

AI-Accelerated payments modernization

Proof of Value — multi-service intelligent development harness

Applied Innovation Exchange - Hyderabad, August 2026

Capgemini

The challenge

Enterprise payment platforms are among the most complex software ecosystems in existence.

1,400+	50–200	3–5 days
Microservice repositories	Services per payment flow	Avg. requirement-to-deploy

- **A single business requirement** — touches multiple services across multiple repositories
- **Tribal knowledge** — determines which services need changes, undocumented, unreliable
- **Cross-service dependencies** — changes in service A break service B's contracts silently
- **Regulatory pressure** — demands auditability, traceability, and provable correctness

Capgemini
APPLIED INNOVATION EXCHANGE

Current state — single-repo AI is solved

✓ What works today

- Read a Jira story
- Connect to a single repository
- Generate code changes
- Create a pull request
- Validate against acceptance criteria

✗ What doesn't exist

- Multi-repo requirement decomposition
- Cross-service contract awareness
- Token-efficient context across 10+ repos
- Architectural pattern enforcement
- Production-grade validation beyond unit tests
- Audit-grade governance for regulated systems

"AI solved code generation speed. The new bottleneck is validation — proving AI-generated code actually works in production-like conditions."

— Signadot, May 2026

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Capgemini
APPLIED INNOVATION EXCHANGE

01

Market landscape & competitive analysis

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

AI coding agents — commercial landscape

Product	Vendor	Key capability	Multi-repo	Gap
Copilot Coding Agent	GitHub / Microsoft	Issue-to-PR workflow, background execution	Per-repo	No cross-service awareness
Devin	Cognition AI	Full autonomous engineer — plans, codes, debugs	Single-repo	Limited orchestration across services
Amazon Q Developer	AWS	AWS-native CI/CD, security scanning, IAM	Workspace	AWS-centric, no payment domain
Augment Code (Cosmos)	Augment	400K+ file Context Engine, semantic deps	Best-in-class	No domain awareness (payments)
Factory AI	Factory	Enterprise agents, custom workflows	Enterprise	Generic — no payment protocol
OpenHands	Open Source	Top SWE-bench performer, autonomous loop	Single-repo	Context exhaustion on large codebases

"No current product adequately handles multi-repository payment-domain context. The closest (Augment Code) provides cross-repo analysis but lacks payment domain awareness, compliance rules, and regulatory governance."

—

The benchmark illusion

SWE-bench Verified (Aug 2026)

Claude Opus 5: ~97%

GPT-5.6 Sol: ~96%

Claude Fable 5: ~95%

Source: SWE-bench Leaderboard, August 2026

"SWE-bench Verified no longer measures frontier coding capabilities — remaining failures reflect dataset properties, not model limits."

— OpenAI Research, August 2026

What reality shows (GPT-4):

88.4%

Single-function (HumanEval)

53.0%

Repo-level with local context

17.4%

Repo-level without cross-file context

Source: DevEval — Li et al., ACL 2024. 1,874 samples, 117 repos

Microservice generation (2025–2026)

- RepoGenesis: 23.7% best Pass@1 — 50.2% failures from cross-file consistency
- RepoCod: <30% — No model exceeds 30% on real repos
- RAL-Bench: <45% — No model exceeds 45% functional score

Sources: RepoGenesis (ACL 2025), RepoCod (ACL 2025), RAL-Bench (Feb 2026)

Why AI coding agents fail in enterprise

The context window cliff

11 of 13 frontier models drop below 50% accuracy by 32K tokens. GPT-4o: 99.3% → 69.7%. [Source: Tian Pan, April 2026]

- **Silent truncation** — framework drops oldest messages, agent loses original instructions
- **Summarization distortion** — 'never exceed 500 req/s' becomes 'rate limiting considerations'
- **Sliding window amnesia** — step 7 contradicts step 2 because step 2 no longer exists

Multi-repo blindness

- **Contract drift** — changes Service A without knowing it breaks Service B's API contract
- **Integration blindness** — unit tests pass, integration tests fail across service boundaries
- **Hallucination compounding** — AI code persists 16% longer, hallucinated bugs become 'ground truth'

"An AI coding bot caused a service outage at Amazon (Dec 2025). Amazon now requires senior engineer sign-off on all AI-assisted changes."

— American Banker, January 2026

Capgemini
APPLIED INNOVATION EXCHANGE

Risks specific to payments & financial services

Regulatory dimension	Risk from AI code generation	Severity
PCI-DSS	AI-generated code handling card data may introduce scope creep, missing encryption	Critical
SOX audit	No clear audit trail for AI-generated logic — 'who authored this?' unanswerable	Critical
AML / KYC	Hallucinated business rules in transaction monitoring create detection gaps	Critical
GDPR / Privacy	Agents may inadvertently process or log PII without consent flows	High
EU AI Act	Code generation in financial decision-making triggers high-risk classification	High
Operational risk	A broken feature in payments can trigger fraud or systemic risk	Critical

2,000+
Exposed AI apps, 40% leaking data

85%
Orgs expect AI to facilitate payments

30%
Tech pros distrust AI code (DORA 2025)

"Financial outputs flow into audited results, requiring precision, traceability, and data security that early AI tools cannot guarantee."

— BCG, May 2026

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Capgemini
APPLIED INNOVATION EXCHANGE

02

Our approach — the AI development harness

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Capgemini
APPLIED INNOVATION EXCHANGE

The AI development harness — concept

> *"Because coding agents cannot be relied on to consistently follow instructions, they must operate within deterministic guardrails."*
>
> — Chris Richardson, microservices.io (March 2026)

A harness is not a coding tool — it's a set of constraints, feedback loops, and verification layers.
Everyone can generate code. The harness ensures you can trust it.

5 **Governance & audit trail**
Risk-tiered review, regulatory traceability, compliance-ready provenance

4 **Deterministic verification harness**
Contract validation, architectural rules, cross-service integration checks

3 **Coordinated multi-service code generation**
Parallel agents scoped to service boundaries, contract-aware changes

2 **Intelligent requirement decomposition**
Requirement → impacted services mapping, dependency analysis

1 **Payment-aware code graph**
Semantic dependency graph with domain knowledge (message formats, flows, integrations)

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Layer 1 — Payment-aware code graph

What it does

- Builds semantic dependency graph across all repos
- Maps domain semantics — message formats, payment flows
- Enables: "If I change this field, which services break?"
- AST parsing + API contract extraction + domain enrichment
- Queryable graph interface for agent context retrieval

Research foundation

- **LogicLens** — semantic multi-repo graph via AST + LLM [paper]
- **GRACE** — multi-level code graph (file, AST, call graph, hierarchy)
- **AST-derived Graph-RAG** — outperforms LLM-extracted graphs [paper]

"Vector-only RAG fails on multi-hop architectural reasoning (controller → service → repository chains). Graph-based retrieval succeeds."

— ArXiv:2601.08773, January 2026

Layer 2 — Intelligent requirement decomposition

A single Jira story says 'add currency conversion support.' Today, tribal knowledge decides which services change. The agent needs to make that same determination — without tribal knowledge.



- **Requirement → service mapping** — queries code graph for affected payment flow stages
- **Dependency cascade** — identifies second-order impacts across services
- **Token-efficient retrieval** — pulls only relevant code (not entire repos)
- **Confidence scoring** — flags uncertainty for human confirmation

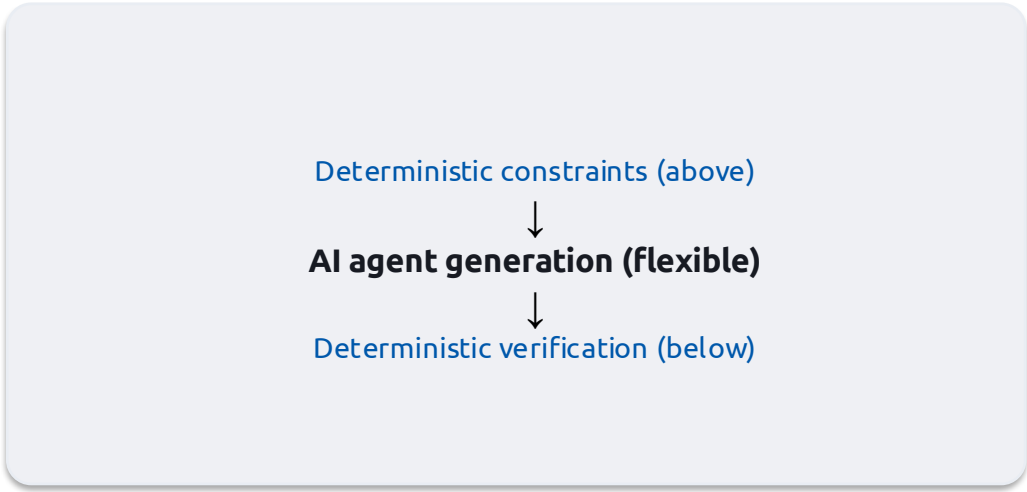
84%	50K→2K	0
Token reduction via memory pointers	Sub-agent isolation compression	Tribal knowledge dependency

Layer 3 — Coordinated multi-service generation

Six proven patterns (Augment Code, 2026)

- Spec-driven decomposition
- Worktree isolation per agent
- Coordinator / Specialist / Verifier
- Model routing (reasoning vs. fast)
- Automated quality gates
- Sequential merge strategies

The sandwich pattern



— Chris Richardson's Coding Agent Sandwich (May 2026)

Applied to payments

- **Coordinator agent** — understands payment flow, decomposes tasks, maintains contracts
- **Service specialists** — scoped to one service each, deep context on that repo
- **Verifier agent** — validates contracts, runs cross-service checks, flags drift

Layer 4 — Deterministic verification harness

The differentiator. The harness catches what agents miss — deterministically, without relying on another AI to verify AI output.

Layer	What it checks	Enforcement
Pre-commit hooks	Linting, type checking, secret scanning, dead code	Deterministic — blocks commit
Contract validation	API contracts (OpenAPI/AsyncAPI) between services	Deterministic — breaks build
Architectural rules	Naming, dependency direction, data flow, message format	Rule engine — rejects violation
Cross-service integration	End-to-end payment flow works after changes	Integration tests — blocks merge

Hallucination detection (AST-based)

100% precision, 87.6% recall — auto-corrected 77% of hallucinations.
Non-executing, deterministic rules for API and identifier conflicts.

> *"The easiest pull requests to approve are often the ones that deserve the closest inspection. AI-generated code arrives well-formatted and convincing."*
>
> — Codacy, July 2026

Layer 5 — Governance & audit trail

Risk-tiered autonomy

Risk	Change type	Approval
Low	Logging, formatting, docs	Auto-merge
Medium	Business logic (non-payment)	Peer review
High	Payment logic, transactions	Senior sign-off
Critical	Compliance-affecting, regulatory	Architecture board

Audit-grade provenance

- Every change linked to originating requirement
- Full chain: requirement → design → code → test → deploy
- AI vs human authorship tagged at commit level
- Immutable audit trail (SOX-grade)
- Override & rollback with justification
- Feature flags for gradual rollout

Regulatory explainability

- "Why was this code generated?" — traceable to requirement
- "What tests validate correctness?" — linked to verification results
- "Who approved deployment?" — governance chain complete

Capgemini
APPLIED INNOVATION EXCHANGE

03

Proof of Value — what we'll build

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Capgemini
APPLIED INNOVATION EXCHANGE

PoV architecture — simplified payment flow

We build a representative payment processing system in our environment — same architectural patterns, simplified scale.

Inbound gateway → Validation & routing → Payment processing → Posting & settlement

Service	Mirrors	Key characteristics
Inbound gateway	Message receipt, schema validation	Receives payment messages, validates format, routes downstream
Validation & routing	Rules orchestrator, sanction check	Business rule execution, external calls (mocked), workflow orchestration
Payment processing	Account lookup, FX, MOP derivation	Core payment logic, partner integrations, account operations
Posting & settlement	Posting entry, outbound message	Creates debit/credit instructions, generates outbound messages

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document

Demo scenario — live walkthrough

Scenario: "Add currency conversion validation before posting" — touches multiple services.

Step	What happens	What it proves
1. Requirement input	Agent reads requirement from a story	Natural language understanding
2. Impact analysis	Queries code graph → identifies affected services	Multi-repo awareness
3. Plan & decompose	Presents: 'Here's what I'll change and why'	Transparency, human-in-the-loop
4. Coordinated generation	Parallel agents generate across 3 services	Multi-agent orchestration
5. Harness validation	Contracts valid ✓, arch rules ✓, integration ✓	Trust without blind faith
6. Harness catches error ⭐	**Contract-breaking change → blocked → self-corrects**	**Guardrails work (wow moment)**
7. PRs created	Coordinated PRs, linked to requirement, classified	Governance, traceability

"The 'wow moment' is step 6 — showing that the system catches its own mistakes. This is what separates a demo from a production-grade platform."

—

Capgemini
APPLIED INNOVATION EXCHANGE

Success criteria & next steps

PoV success criteria

- **Multi-repo awareness** — correctly identifies impacted services
- **Coordinated changes** — maintains contract compatibility
- **Harness catches errors** — blocks invalid changes before merge
- **Token efficiency** — operates within reasonable budgets
- **Audit trail** — every change traceable to requirement
- **Speed** — requirement to validated PRs in minutes

Proposed timeline

Week 1–2

Foundation

Build demo services, repos, contracts, CI/CD

Week 3–4

Code graph & agents

Dependency graph, multi-agent orchestration

Week 5–6

Harness & governance

Verification harness, contract validation, audit

Week 7–8

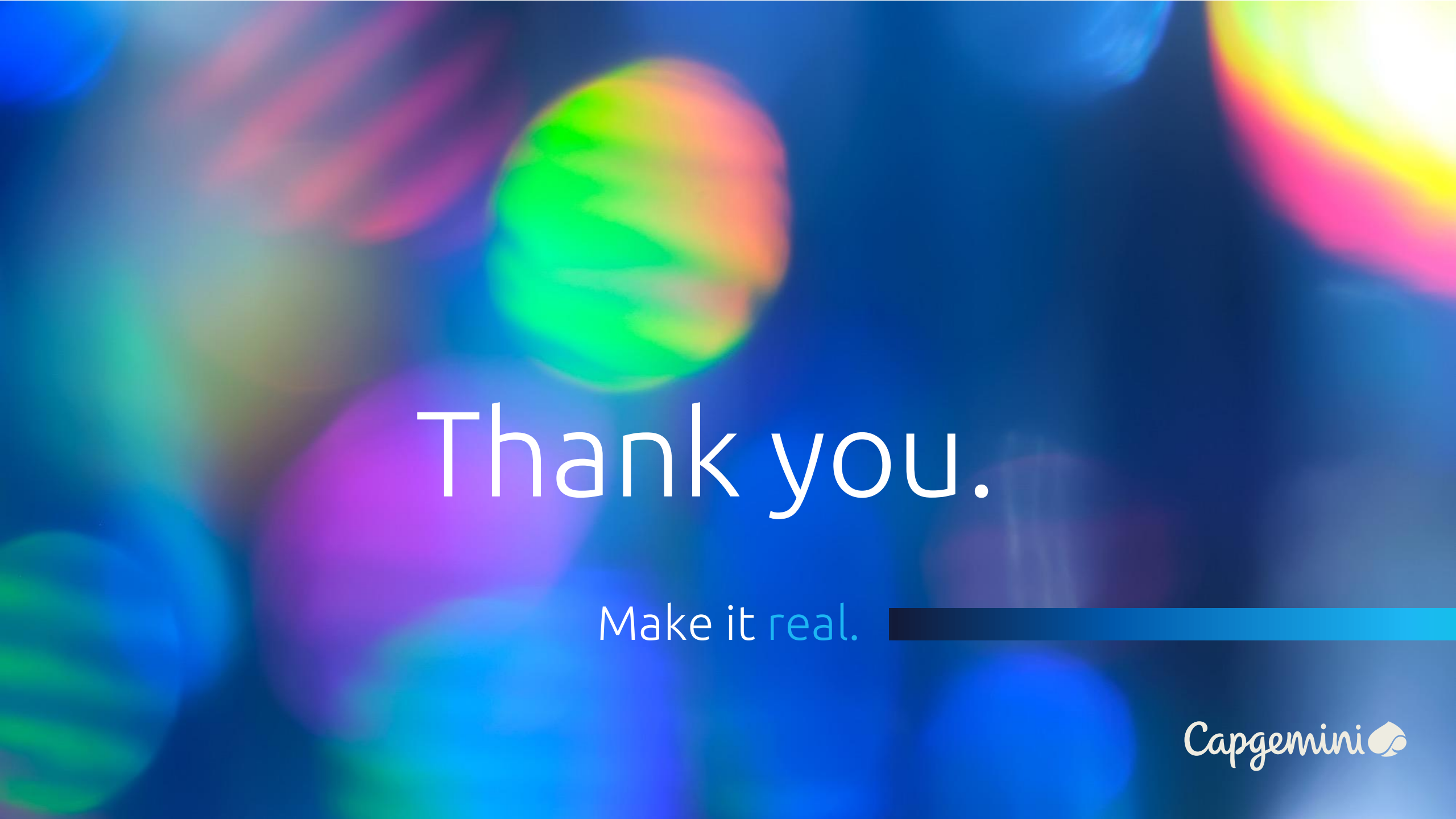
Demo & refinement

End-to-end flow, edge cases, presentation

Immediate next steps

- Align on PoV scope and timeline
- Define demo service architecture
- Begin foundation build

COMPANY CONFIDENTIAL
SEC 1
Capgemini Document


Thank you.
Make it real.
Capgemini